



Hybrid Parallelization of Particle in Cell Monte Carlo Collision (PIC-MCC) Algorithm for Simulation of Low Temperature Plasmas

Bhaskar Chaudhury^{1(✉)}, Mihir Shah¹, Unnati Parekh¹, Hasnain Gandhi¹,
Paramjeet Desai¹, Keval Shah¹, Anusha Phadnis¹, Miral Shah¹,
Mainak Bandyopadhyay^{2,3}, and Arun Chakraborty²

¹ Group in Computational Science and HPC, DA-IICT, Gandhinagar 382007, India
bhaskar_chaudhury@daiict.ac.in

² ITER-India, Institute for Plasma Research (IPR), Gandhinagar 382428, India

³ Homi Bhabha National Institute (HBNI), Anushakti Nagar, Mumbai 400094, India

Abstract. We illustrate the parallelization of PIC code, for kinetic simulation of Low Temperature Plasmas, on Intel Multicore (Xeon) and Manycore (Xeon Phi) architectures, and subsequently on a HPC cluster. The implementation of 2D-3v PIC-MCC algorithm described in the paper involves computational solution of Vlassov-Poisson equations, which provides the spatial and temporal evolution of charged-particle velocity distribution functions in plasmas under the effect of self-consistent electromagnetic fields and collisions. Stringent numerical constraints on total number of particles, number of grid points and simulation time-scale associated with PIC codes makes it computationally prohibitive on CPUs (serial code) in case of large problem sizes. We first describe a shared memory parallelization technique using OpenMP library and then propose a hybrid parallel scheme (OpenMP+MPI) consisting of a distributed memory system. OpenMP based PIC code has been executed on Xeon processor and Xeon-Phi co-processors (Knights Corner and Knights Landing) and we compare our results against a serial implementation on Intel core i5 processor. Finally, we compare the results of the hybrid parallel code with the OpenMP based parallel code. Hybrid strategy based on OpenMP and MPI, involving a three-level parallelization (instruction-level, thread-level over many cores and node-level across a cluster of Xeon processors), achieves a linear speedup on an HPC cluster with 4 nodes (total 64 cores). The results show that our particle decomposition based hybrid parallelization technique using private grids scale efficiently with increasing problem size and number of cores in the cluster.

1 Introduction

The Low Temperature Plasma (LTP) science and technology is a continuously growing interdisciplinary research area with broad range of applications [1].

© Springer Nature Singapore Pte Ltd. 2019

A. Majumdar and R. Arora (Eds.): SCEC 2018, CCIS 964, pp. 32–53, 2019.

https://doi.org/10.1007/978-981-13-7729-7_3

To aid the experimental investigations/developments in this area, several kinetic and fluid simulation models have been developed to study this complex medium consisting of electrons, ions, neutral atoms and molecules. Accurate simulation of plasmas using a kinetic model such as Particle-In-Cell (PIC) method plays an important role in the development of various plasma technologies [2]. PIC-MCC (Particle in Cell Monte Carlo Collision) method is a computationally expensive technique to investigate plasmas, and provides spatial and temporal evolution of the charged-particle velocity distribution functions under the effect of self-consistent electromagnetic (EM) fields and collisions [3,4]. In this method, we follow the trajectories of a representative number of charged particles in the phase space on a mesh free Lagrangian grid and take into account the collective interaction of the particles by solving Poisson's equation using an fixed point Euler grid [2-4]. The random sampling based Monte-Carlo Collisions (MCC) method is used to calculate the charged particle collisions with neutral atoms in PIC-MCC algorithm [2].

The dimensions of the phase-space, grid size, number of computational particles, time-step and total number of iterations required for the simulation [4] determines the computational cost of PIC codes. Compliance of several numerical constraints such as minimum grid spacing, minimum time step and minimum number of particles per cell (PPC) are often required for accurate simulation [2,4,5]. Typical run-times for PIC simulations ranges from several hours to months and requires advanced HPC facilities. The design and development of new high performance PIC codes for the exascale computing systems are topic of several recent investigations [6-8]. The paper by Hariri et al. describes the implementation of an efficient PIC simulation using OpenACC platform and reports about different parallelization strategies inspired by the newer technology in large scale computation [6]. Other major work which has benefited from the latest technological advancements includes development of PIC code by Derouillat et al. [7]. These works demonstrate the challenges in parallelization/optimization of PIC algorithm and why parallelization of PIC codes for emerging computing architectures is an important/challenging research problem.

Over the past several years, much effort has been put in to development of efficient parallel PIC codes for plasma simulation [9]. PIC code scaling for the 32 K nodes of Cray XT3 supercomputer using Fortran and MPI implementation have been reported in [10]. PIC-on-GPU [11], is a very well known open source implementation based on the Nvidia CUDA toolkit, which can scale to a GPU cluster if needed. Several strategies for parallelization of PIC codes on GPUs, for simulations of plasmas, have been reported recently. More details related to parallelization and optimization strategies of PIC codes reported in some of these works can be found in [4,6,8,12] and the references therein. The code developed by Decyk et al. [8] has different versions of the simulation specific to the architectures on which they are executed. The authors implement two new strategies for GPU, and suggested that highly efficient OpenMP implementation of the PIC code on core i7 shall work similarly on upcoming MIC architectures from Intel. In the recent article by Madduri et al. [13], the authors have presented

an extensive comparison between various architectures for the computationally intensive charge deposition module. They have demonstrated the improvements arriving due to the newly developed methods of grid decomposition and synchronization against the existing strategies. The authors also extend this approach to the next generation of multi-core architectures.

The aim of this paper is to report about the development of an efficient double precision 2D-3v parallel PIC code optimized for the Intel multicore and manycore architectures, and subsequently a hybrid code (MPI+OpenMP) which runs on a CPU based cluster. Our work involves the development of a serial PIC-MCC code from scratch and validating it with published results from [14]. In this paper, firstly, we discuss the strategies used for efficient parallelization of the PIC code using openMP on Xeon processor and Xeon-Phi co-processors (Knights Corner and Knights Landing) and we compare our results against a serial implementation. Secondly, we describe the strategy for the hybrid implementation (using MPI+OpenMP) and we compare the performance results against the shared-memory based parallel code. Finally, we report and study the effect of simulation parameters such as grid size, number of particles and PPC on the run time of the hybrid code. The emergence of new hardware with more cores, better memory hierarchy and additional features such as vectorization is singularly not sufficient for solving the computational challenges of PIC simulations. The development of parallel algorithms needs to progress accordingly in order to achieve sustainable results. We focus our experimentation on the two generations of Xeon phi architectures, along with the Xeon processor for comparisons. However, because of high computational intensity and long simulation time, we need to develop a hybrid code which can be used to analyze larger problem sizes over longer time scales.

The paper is organized as follows: The computational model (PIC-MCC algorithm) is briefly described in Sect. 2. Section 3, gives an overview of the different architectures used in this work. Section 4 highlights the implementation details specific to our serial/parallel PIC-MCC code and a typical case study used for the validation of the code for modeling and simulation of a LTP experiment. Different parallelization strategies and optimization techniques are discussed in Sect. 5. The detailed performance analysis and important observations regarding performance results is given in Sect. 6, followed by conclusions in Sect. 7.

2 Computational Model

PIC is a self-consistent method wherein motion of charged particles is solved in the presence of self-consistent fields as well as externally applied EM field. When the current generated by plasma is low, the self induced magnetic field can be ignored and it is known as Electrostatic (ES) PIC method. The flowchart of self-consistent ES PIC-MCC method is shown in Fig. 1. The algorithm consists of following the trajectories of a representative number of charged particles in the phase space on a Lagrangian grid (mesh free) and describing the collective interaction of the particles by solving Poisson's equation using an Euler grid

(fixed grid point) [3,4]. It involves splitting of the kinetic Vlasov-Poisson equation (Eq. (1)) into two ordinary differential equations (Eqs. (2) and (3)).

$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \frac{\partial f}{\partial \mathbf{x}} + \frac{q}{m} \cdot \left(\mathbf{E} + \frac{\mathbf{v} \times \mathbf{B}}{c} \right) \cdot \frac{\partial f}{\partial \mathbf{v}} = \left(\frac{\partial f}{\partial t} \right)_{\text{colli}} \quad (1)$$

$$\frac{d\mathbf{r}}{dt} = \mathbf{v} \quad (2)$$

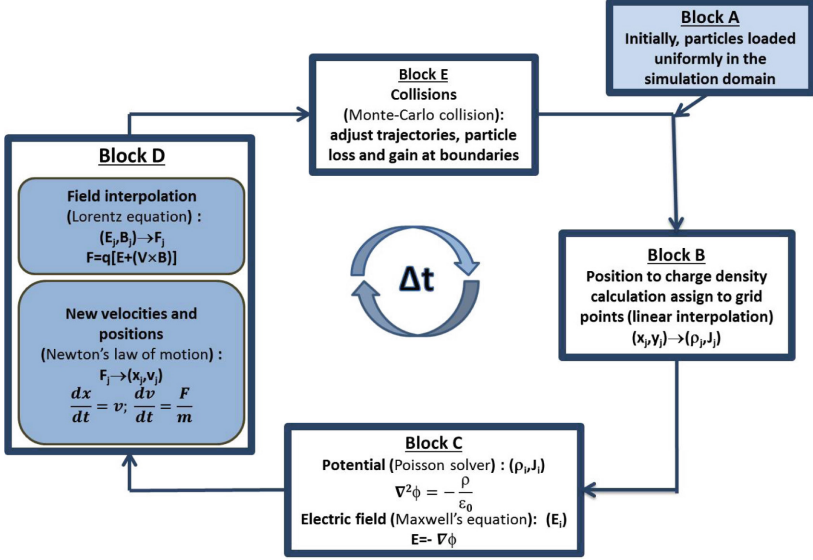


Fig. 1. Flowchart of the ES PIC-MCC algorithm [4]

$$\mathbf{F} = m \frac{d\mathbf{v}}{dt} = q \cdot \left(\mathbf{E} + \frac{\mathbf{v} \times \mathbf{B}}{c} \right) \quad (3)$$

The simulation domain is divided in spatial grids with grid size small enough (smaller than debye length) to resolve the variation in plasma parameters [3]. Initially, computational particles (representing electrons and ions) are uniformly loaded inside the simulation domain. Their initial positions and velocities are assigned randomly [15,16] (block A of Fig. 1). Maxwellian distribution is used for initialization of particle velocities.

Charge density is calculated on grid points from Lagrangian phase space information using linear interpolation weighing scheme (block B in Fig. 1) [4]. This is a computationally expensive part of the PIC-MCC method. A particle contributes its charge to the surrounding four points and this is accomplished by performing interpolation at every iteration for all particles as shown in Fig. 2. We

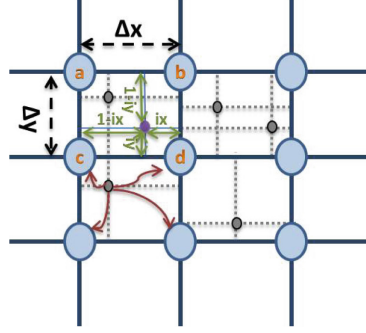


Fig. 2. Charge density assignment on grid points using linear interpolation. A, B, C and D are grid points on the 2-D simulation grid. Green and black circles are particles inside different grid cells [4]. (Color figure online)

have used a linear interpolation scheme which is less noisy, and computationally cheaper compared to zeroth order and higher order schemes [3]. For example, contribution of a particle (shown in green in Fig. 2) towards charge density (ρ_A) on grid point A is calculated using the following equation,

$$\rho_A = q \left(\frac{ix \cdot iy}{\Delta x^2 \cdot \Delta y^2} \right) \quad (4)$$

where Δx and Δy are grid sizes, and ix , iy are fractional distances of particle from the cell origin in the x and y direction respectively.

Scalar potential from charge densities is calculated using Poisson's equation Eq. (5). An optimized, high performance and memory efficient PARDISO library is used to solve Poisson's equation [17]. Electric field E on grid points is derived by using finite difference form given in Eq. (6) (block C of Fig. 1).

$$\nabla^2 \phi = -\frac{\rho}{\epsilon_0} \quad (5)$$

$$\mathbf{E} = -\nabla \phi \quad (6)$$

New velocities and positions of particles are calculated using the newly computed field on Euler-grids. Euler grid to Lagrangian node transformation is accomplished by linear interpolation which is similar to the interpolation scheme described above for charge deposition. Force given by Eq. (3) is used to update equations of motion (Eqs. (2) and (3)) by using Boris method [18]. Mover (block D of Fig. 1) is also computationally intensive due to calculation of position and velocity of all the particles.

Finally, Monte-Carlo collisions (MCC, block E of Fig. 1) are applied to randomly selected particles using collision probability [16]. This method is used to select type of the collision and calculate the new particle velocities after collision [12].

The earliest works in the PIC code started with the development of 1-D code which was initially done to study the fluid dynamics calculations. The original code found various applications because of its efficient implementation in the massively parallel computers. This initial work laid the foundation of first electrostatic plasma model by the Buneman's group at Stanford, and Birdsall and Bridges group at Berkeley [3]. Advancements in high performance and distributed computing have expanded the applicability of PIC method to broader spectrum of plasma science and technology [19].

3 Hardware and Programming Frameworks

Large scale highly parallel systems based on shared memory architectures are dominant computing platforms that give rise to many different parallel programming paradigms. **OpenMP** is a popular API for parallel programming on shared memory architecture. It uses a set of compiler directives (pragmas), library routines and environment variables to specify multi-threaded execution of the code. On distributed memory systems, **MPI** is widely used for writing message passing programs across nodes of a cluster. OpenMP based code can be also ported to emerging many-core architectures such as Intel Xeon-Phi [20]. Xeon Phi is a series of many-core processors. This vector processor has 256 bit wide SIMD registers and multiple cores. Its special vector processor helps in doing SIMD instructions faster. Depending on the version, it can do 4 or 8 double precision operations in the same time as a single operation on conventional processors. It supports four way multi-threading and has high bandwidth for on chip data access, which brings down latency to a great extent.

For small problem sizes, researchers generally use an OpenMP based code, however for bigger problem sizes and better accuracy we need hybrid codes. Thus, a combination of shared memory and message passing parallelization paradigms within the same application, often known as hybrid programming, is expected to provide more efficient parallelization strategy for clusters of shared memory nodes. The **hybrid OpenMP+MPI** programming refers to a programming style in which communication between nodes is handled by MPI processes and each MPI process has several OpenMP threads running inside to occupy the CPUs for computation.

Table 1 shows the four test benches which have been used in our work. The reference benchmark is a standard standalone CPU with 2 cores. But the hardware limitations of this CPU limits the grid sizes for which our simulation can be run. Thus, the serial code is parallelized using OpenMP and a high degree of thread level parallelism is achieved when it is run on the many-core architecture of the Intel Xeon Phi processors. When we develop the hybrid code, we run it on the 4 node cluster with shared memory multi-core processors, such that we get advantage of node-level parallelism as well.

Table 1. Specifications of the platforms used in our computational investigation. The compilation was done through the Intel Compiler 16.0.3 on all architectures. The compilation requires `-fopenmp`, `-lpthread`, `-lm` and `-ldl` flags. Other flags included are `libiomp5.a`, `libmkl_blacs_openmpi_ilp64.a`

	Reference	Bench 1	Bench 2	Bench 3	Bench 4
CPU	Intel i5-4210U Processor	Intel Xeon 2630, 2 sockets (Xeon)	Intel Xeon Phi 5110P Co-processor (Phi 5)	Intel Xeon Phi 7250 Processor (Phi 7)	4 node cluster: 2 Intel processor (E5-2640)/node
Frequency (GHz)	1.70	2.40	1.053	1.40	2.60
Cores	2	16	60	68	64
L2 size (MB)	0.25 (per core)	0.25 (per core)	30 (shared)	34 (shared)	-
L3 size (MB)	1.5 (per core)	20 (shared)	-	-	20 (shared)
Peak Memory Bandwidth (GB/s)	25.6 (2 channels)	59 (4 channels)	320 (16 channels)	115.2 (6 channels)	25.6 (2 channels)

4 Implementation Details and Code Validation

In this section, we present the details of the primary data structures used in our code. We also present the analysis of the code in terms of the arithmetic intensity in important subroutines and the execution time share of each subroutine followed by a case study for the validation of the code.

4.1 Data Structures

There are two essential groups of quantities associated with PIC simulations: Particle quantities and Grid quantities. As explained earlier, the particle to grid interpolations (for charge deposition) and the grid to particle interpolations (for moving the particles) are performed extensively. Hence, these quantities interact with each other very often.

In our $2D - 3v$ PIC code, for a specific computational particle representing many actual particles, particle quantities are the phase space information (position and velocity vectors) of that particle along with an identifier describing the species of that particle (electron or ion). These quantities are grouped into a structure which is named as *Particle* and to keep track of these quantities for every particle, an array of *Particle* structure is used as shown in Fig. 4(c).

Quantities such as charge density, potential, electric field, magnetic field have been discretized over the 2D grid. Since these quantities are recorded for every grid point, we have called them grid quantities. Each grid point has two components of electric field, one along X and the other along Y direction. Hence,

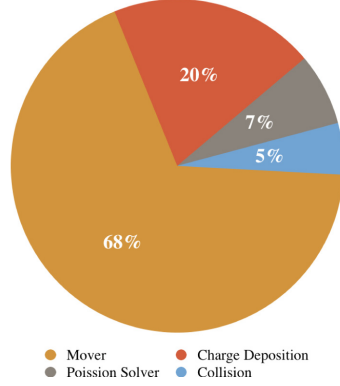


Fig. 3. The contribution of different modules/subroutines towards the total run-time of the serial code for a typical simulation (moderate problem size) [4]. Grid size of 1024×1024 with 20 PPC density simulation. Executed on a single core of Intel Xeon E5-2630 v3 with 64 GB of DDR3 RAM. *Mover* and *Charge Deposition* prevail the overall execution time.

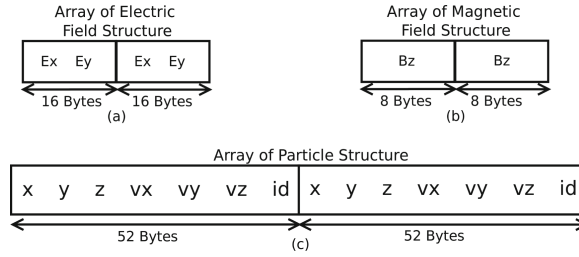


Fig. 4. Visualization of particle and grid data structure. (a) Electric field components are stored as an array of structure. (b) Magnetic field has only one component, so it is stored in an array of double data type. (c) Every particle has their own phase space information stored in a Particle structure. Hence, information of all particles is stored as an array of structures [4].

electric field information is stored in a structure for a particular grid point and an array of these structures is used to maintain the information of electric field for the whole grid. The magnetic field for our simulation purpose has been kept varying only in the Z direction. Hence, only the array of values is needed to keep track of magnetic field over all grid points. Same is the case for charge density and electric potential. All the quantities (particle and grid) are stored as double precision floating point values during the simulation. An important point to note here is that even though the grid quantities are over the grid points of a 2D grid, but when storing it as an array, we have used just a 1D array rather than using the 2D array (array of arrays) [4].

4.2 Arithmetic Intensity

¹Arithmetic intensity of a section of code (or subroutine) is the ratio of the number of computations performed to the number of bytes transferred from/to memory in that code section and gives an idea about whether the section (module or subroutine) is compute or memory bound. From the Fig. 3, we can see that the most of the simulation time goes into *Charge deposition* and *Mover* module. Hence, the important subroutines are charge deposition and mover. The arithmetic intensity of these important subroutines are presented in Table 2 and from that we can see that both *Mover* and *Charge deposition* are memory bound.

Mover needs to load 6 phase space values of 8 bytes each and another 4 bytes are needed to identify the type of particle – id attribute of Particle structure as shown in Fig. 4(c). Based on the type of particle, we have to fetch the charge to mass ratio for that particle which requires 8 bytes to be fetched. 12 values corresponding to E_x , E_y and B_z on four cell corners (8 bytes each) are read to interpolate the fields on to the particles as shown in Fig. 2. Finally, all the 6 phase space values are updated (8 bytes each). The rest of the data is for keeping track of the different particle types and their start index in the particle data structure.

In *Charge Deposition*, x and y coordinates of the particle along with an identifier for the type of particle is loaded (20 bytes total). Weighing factor (8 bytes) is required to balance the charge density against the actual number of particles. Finally, the charge density is loaded (8 bytes each) and updated (8 bytes each) on all the four cell corners. Rest of the data is used for book keeping.

Table 2. Arithmetic intensity (ratio of number of computations to the number of bytes transferred from/to memory) [4].

	Mover	CD
Double-precision operations (FLOP)	91	19
Double-precision Read/Write (Byte)	204	92
Arithmetic intensity (FLOP/Byte)	0.446	0.206

4.3 Code Validation and Comparison of Simulations with Experiment

One of the important application of PIC-MCC method is in investigation of Low Temperature Plasmas involving transport of charged particles across magnetic field. The physics of plasma transport across strong magnetic field is a complex phenomenon and present in many applications including fusion reactors, thruster and also in negative ion sources. We have used this standard problem for validating our PIC-MCC code [12, 14] and particularly to simulate the collisional transport across magnetic filters under conditions similar to real ROBIN

¹ The text in this particular subsection has been adopted from [4].

negative ion source experiment. ROBIN (Rf operated Beam source in India for Negative ion research) has been setup at IPR, Gandhinagar to understand and investigate the different issues related to production, transport and extraction of negative hydrogen ions in negative ion sources for fusion applications [21]. The source consists of a driver, an expansion chamber, a transverse magnetic filter field (TMF) and extraction grids as shown in Fig. 5. Simulation domain, considered for this study, takes into account 2-D displacements of particles in X-Y plane with 3 velocity components under the influence of an externally applied magnetic field (B_z) perpendicular to the simulation domain (in z direction with Gaussian shape variation in x direction [14]). The spatial grid size in the computational domain is small enough to resolve the important physical length scales such as electron Debye length. Temporal time steps are chosen such that it can resolve important time scales of the system and is generally given by Courant-Friedrichs-Lewy (CFL) condition $\Delta t \leq \frac{0.2}{\omega_p}$. Using this code (number of grid points = 800×154 , grid size = $6.5\text{E}-4\text{ m}$, time step = $1.2\text{ E}-10\text{ s}$, initial number of electrons and ions = 2.5 million each), we have investigated different plasma characteristics such as plasma potential, electron temperature, electron and ion density profiles, electric field, current etc. (as shown in Fig. 5) in the case of a Gaussian shaped magnetic filter, as a case. Simulation results show similar qualitative and quantitative behaviors as observed during the first phase of ROBIN experiments.

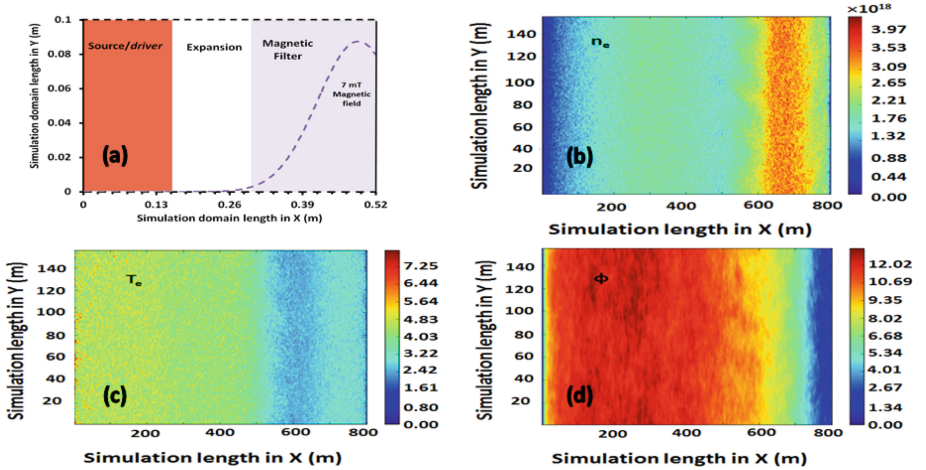


Fig. 5. (a) Schematic of the computational domain. Sub-figures (b, c, d) shows the profile of the electron density, electron temperature and potential at $72\mu\text{s}$ of physical time.

5 Parallelization Strategy

The parallelization of PIC code serves as a major research problem in the area of High Performance Computing. The earliest works on parallelizing the PIC codes were designed for distributed memory systems. The nodes simply communicate using message passing, no shared memory is available to take advantage of thread level parallelism [22]. This approach does not scale for larger problem sizes [8]. With the advancements of shared memory parallelism, researchers tried to implement PIC code using OpenMP library. This method also suffers from similar problem of scalability [7].

Newer methods for parallelizing PIC codes are based on distributed systems with shared memory parallelization in each node. This system uses the hybrid MPI+OpenMP programming technique which is the focus of this paper. There are two ways in which hybrid parallelization is achieved. These are particle and grid based decomposition.

The particle-based decomposition is widely used because of its easy implementation and near perfect load balance [7]. The hybrid programming paradigm used in this paper implements OpenMP based thread level shared memory parallelization inside MPI based node level processes, i.e., each MPI node has a fixed number of OpenMP threads and which communicate among MPI processes outside the parallel region.

The grid-based decomposition is based on sharing a part of grid among different nodes rather than the particles. This method suffers from the problem of load balancing as it is difficult to ensure that the particles are distributed equally among all nodes. A similar method consisting of very fine-grain domain decomposition referred to as patch-based decomposition has also been used in recent works [7], where patches denote very small sub domains. This method is an extension of grid-based decomposition.

5.1 Shared Memory (OpenMP) Parallelization Strategy

Parallelization of Charge Deposition Module. There are two different strategies which can be used to parallelize the *Charge Deposition* module.

(1) Dividing the rows of the grid equally among the threads. The interpolations of all the particles lying in one region will be done by the thread which is assigned that region. If each thread shares the entire grid then the two threads having common grid points along the common rows are vulnerable to race conditions. Therefore we must give each thread an exclusive portion of the whole grid. To get a consistent view of grid quantities, we need to aggregate the grid quantities shared between two threads at the boundary. Due to the continuous movement of the particles in the computational domain, it is very likely that there are unequal number of particles across the different areas of the grid and hence unequal number of particles per thread which may lead to **uneven load balancing**. Moreover, each thread must exactly know the particles that are lying in the region assigned to it along with the location of those particles in the array of particles. To do this efficiently, the particles lying in one region have to be

grouped together in the array of particles. The start and end index of that group has to be calculated. This grouping has to be done after every iteration since the particles will be moving continuously throughout the simulation. Doing this at every iteration introduces extra overhead and complexity.

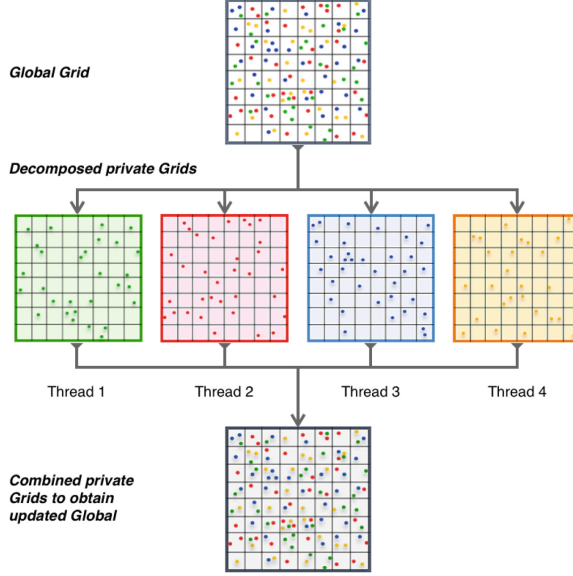


Fig. 6. Parallelization strategy for charge deposition module.

(2) Dividing equal number of particles among the threads. This strategy is motivated by the Principle of Superposition. Since the interpolations in this module applies as particle-to-grid, the race conditions may arise when two different particles handled by two different threads try to update the grid quantities pertaining to the same grid point simultaneously. In order to avoid race conditions we shall assign each thread a private copy of the global grid as portrayed in Fig. 6. This ensures that each thread works independently on its share of particles without any critical region or requirement for locks. Once the threads have done their share of calculating grid quantities on their own private grids, we aggregate all the private grids to get back the global grid. This strategy involves the overhead of generating private grids for each thread. Synchronization is straightforward.

Out of these two strategies, the second approach would result in efficient load balancing among all the threads as compared to the first one. Also, if we increase the problem size, the latter strategy is scalable since the time taken to synchronize the whole grid is negligible compared to the time taken to perform particle to grid interpolations.

We have used the second strategy in our implementation and the following is the **pseudo-code for parallel execution of charge deposition module**.

```

1 void chargeDeposition()
2   GridQty *private_grid_qty = malloc(no_of_threads * GRID_DIMENSIONS);
3   /* Parallelization over particles using private grids */
4   #pragma omp parallel for
5   for(i = 0; i < total_particles; i++)
6     int id = omp_get_thread_num();
7     /* Position of neighboring grid points */
8     double x1, y1, x2, y2, x3, y3, x4, y4;
9     /* Calculate the position of neighboring grid points
10      based on the position information from particle_array[i] */
11     double update1, update2, update3, update4; /* Updates */
12     /* Calculate the value of updates */
13     /* Update the private grid quantity's value at nearby grid points */
14     private_grid_qty[id * GRID_DIMENSIONS + x1 * Nx + y1] += update1;
15     private_grid_qty[id * GRID_DIMENSIONS + x2 * Nx + y2] += update2;
16     private_grid_qty[id * GRID_DIMENSIONS + x3 * Nx + y3] += update3;
17     private_grid_qty[id * GRID_DIMENSIONS + x4 * Nx + y4] += update4;
18     /* Combine the private grids to form the updated global (Synchronization) */
19     for(i = 0; i < no_of_threads; i++)
20       for(j = 0; j < Nx * Ny; j++)
21         global_grid[j] = private_grid_qty[i * GRID_DIMENSIONS + j];

```

Parallelization of Mover Module. The *Mover* subroutine can be parallelized by dividing the particles equally among the threads. In this subroutine, the interpolations are performed from the grid to particle. Since particles are divided among the threads, a particular particle's phase space information is updated only once by one thread and to do that update, the thread has to just read the grid quantities. Therefore, there will be no race conditions while updating the phase space information of particles in parallel. Hence, this is an embarrassingly-parallel challenge. Moreover, dividing particles equally among the threads will lead to efficient load balancing with a straightforward implementation using static scheduling.

```

1 void mover()
2   /* Parallelization over particles using private grids */
3   #pragma omp parallel for
4   for(i = 0; i < total_particles; i++)
5     double updateX, updateY; /* Updates in position */
6     double updateVx, updateVy, updateVz; /* Updates in velocities */
7     /* Calculate the value of updates based on the value of
8      grid quantities at neighboring grid points */
9     /* Update the phase space information at particle_array[i] */
10    particles_array[i].x += updateX; particles_array[i].y += updateY;
11    particles_array[i].vx += updateVx; particles_array[i].vy += updateVy;
12    particles_array[i].vz += updateVz;

```

Sorting. It has been well established that sorting significantly increases simulation speeds of PIC and eliminates cache thrashing [23]. Cache thrashing occurs while interpolation of quantities from the grid to the particles and particles to grid. If the particle array is sorted based on their positions in the grid, the grid quantities are no more accessed randomly and hence this strategy successfully achieves cache re-usability.

We performed experiments on different architectures to inspect when sorting leads to increase in speedup. The results can be seen in Fig. 7(a). For processors with smaller cache sizes, the grid data structures used in the simulation cannot fit

in the cache completely and thus, sorting helps in improving their performance. But for systems with very large cache size like in the Intel Xeon Phi co-processors and those commonly used today, the grid data structures are able to easily fit in the cache. Thus, sorting adds unnecessary overhead in this case and leads to lower speed up than expected. Therefore, we are not sorting the particles before the charge deposition module.

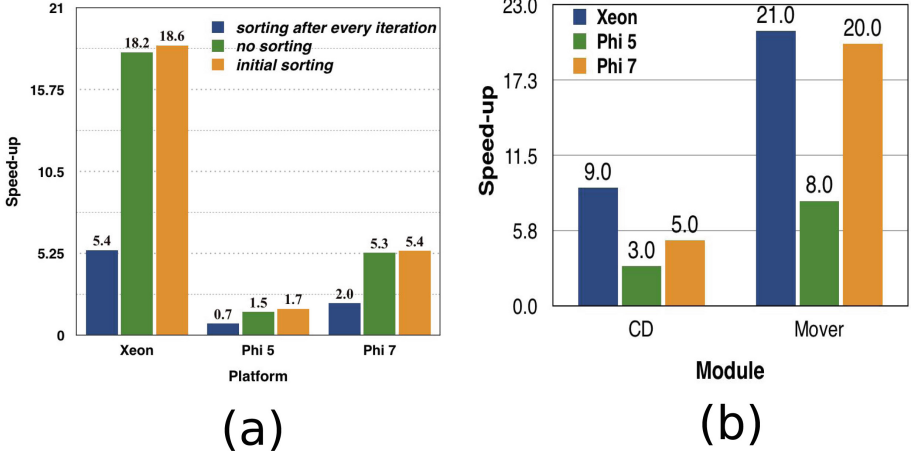


Fig. 7. (a) Strategy-wise maximum speed-up of optimized parallel execution on 3 different architectures relative to a non-sorted serial execution on the Intel i5 processor. (b) Module-wise maximum speed-up on selected architectures (different number of threads on different architectures). This value is calculated relative to a serial execution on an Intel i5 based machine.

Table 3. Cache hit rate comparison (perf analysis). This observation is in accordance with the fact that Phi-7 architecture has a significantly larger cache than the Xeon architecture (Table 1).

	Serial	Parallel
Xeon	88.24	76.22
Phi 7	95.66	86.88

The numerical results in Fig. 7(b) and the supporting figures have been obtained through guidelines laid down by Hoeftler [24]. For analyzing the behavior of strategies on varying architectures, we chose a base architecture (Intel i5) to act as a common reference (Table 1).

There is an extensive variation in the sequential code execution time of the architectures that we have considered for analysis. The values range from 140 s on Intel Xeon E5-2630 (**Xeon**), 386 s on Intel i5, 971 s on Intel Xeon Phi 7250 (**Phi 7**) and 3892 s on Intel Xeon Phi 5110p (**Phi 5**). The simulation corresponds to

a physical time of 18 ns on a square grid of 128-128 partitions with a density of 20 particles per cell. Due to this significant difference in the serial processing power; theoretically, a perfectly parallel code executed on Xeon Phi 5110p with 28 cores will perform at par with the same serial version executed on E5-2630.

Xeon Phi has a more primitive prefetching, slower clock frequency, in-order processing, a smaller pipeline, one instruction per two cycles etc. Therefore, the key to best performance is doing the best optimization in all the three aspects: vectorization, parallelization, and memory utilization. The under-performance of our execution can be explained through the very low vectorization potential of the PIC algorithm as supported by the arithmetic intensity values in Table 2. This explains why the scalar code running on a single thread of Xeon Phi is slower than scalar code running on a Xeon. This is a problem because the scalar unit of Phi 5 is quite under-powered, both for itself and relative to modern processors. This was improved in the Phi 7 but it still remains slower in comparison with the multi-core Xeon.

Inferring from the values in Fig. 7(b), the performance of the overall optimization is significantly specific to the features and ability of the hardware. The clock speed of a single processor, cache size and cores are among the major contributors in shaping of the above results.

5.2 Hybrid Parallelization Strategy

For system architectures enabled for hybrid parallelization, we can make use of multiple nodes connected to each other via the network. We use the MPI library to enable communication between the nodes (*Node-level parallelism*). Each node has a specific number of cores and each core, in turn, is capable of launching hardware threads (*Thread level parallelism*).

Let us revisit the parallelization strategies discussed in Sect. 5.1 to extend them to the hybrid implementation. The parallelization of the *Charge Deposition* module has been achieved using OpenMP. The particles are equally divided among the OpenMP threads and each thread has its own private copy of the grid quantities to avoid race conditions when performing particle to grid interpolation. Once all the OpenMP threads have updated their private copies of the grid quantities, these private copies are aggregated to get the updated global grid. This ensures that the grid quantities are consistent before and after the *Mover* module is executed. For the parallelization of the *Mover* module, the particles are equally divided among the OpenMP threads, as a solution to an embarrassingly parallel sub-problem.

The hybrid parallelization strategy naturally builds from the strategy used for thread level parallelization which is summarized above. At the beginning of the simulation, the particles are equally divided among the nodes as shown in the Level 1 parallelization of Fig. 8. Each node is responsible for executing the *Charge deposition* and *Mover* module for the set of particles that are assigned/private to that node. Since the particles are divided among nodes, each node is required to have its own private copy of the grid in order to avoid race conditions with the other nodes during the *Charge deposition* subroutine. After the execution of the

Charge deposition module, the private copies of each node must be aggregated to get consistent view of grid quantities on global level.

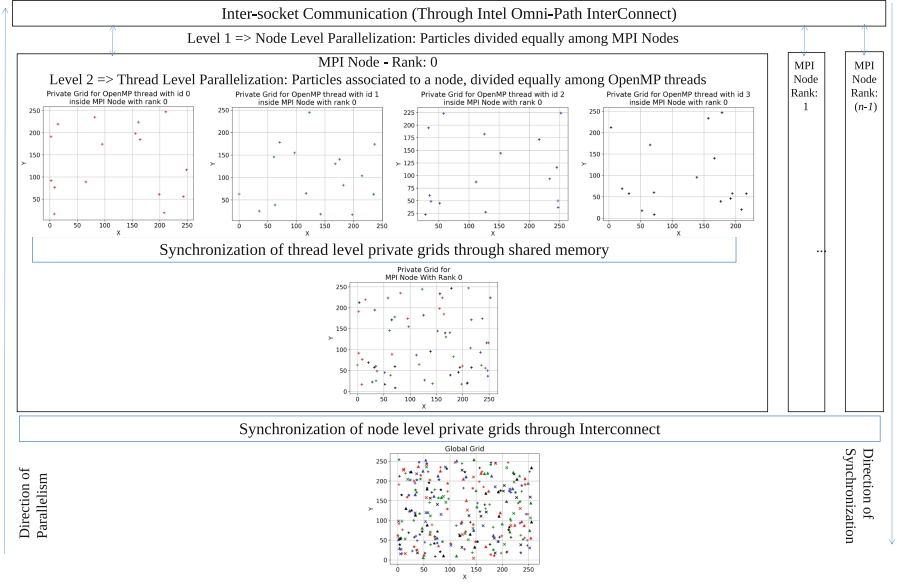


Fig. 8. Visualization of the hybrid parallelization strategy. Level 1 (node-level parallelization): particle distribution per node and the aggregation of private grids of all nodes and Level 2 (thread level parallelization inside a node): further distribution of particles of a node per thread and the synchronization of private grids per thread to get a consistent private grid per node

Since each node has multiple cores, it shall execute the thread level parallelization strategy explained in Sect. 5.1 for the *Charge Deposition* module for the particles assigned to that node to get the updated node-level grid quantities. Once each node has its own private copy of grid quantities updated, each node takes part in the global node-level aggregation of the grid quantities through its own private grid quantities.

For this global aggregation, each node must use the network and communicate to every other node its private copy of grid quantities. Post this communication between nodes, the grid quantities are consistent among all the private node grids. Since, this aggregation takes place over the network, it is expensive communication. It is important to reduce this cost, by optimizing it using the `MPI.Allreduce` function provided by MPI library.

5.3 Pseudo-code of Hybrid Parallelization Strategy

At the beginning of the PIC simulation, the data about initial conditions of the simulation is read from a file. That file contains information about the total

number of computational particles in the simulation. Each MPI node reads the file and is allotted the number of particles that it is supposed to handle as follows:

```

1  int particles_for_this_node = total_particles / MPI_WORLD_SIZE
2  if (mpi_world_rank == MPI_WORLD_SIZE - 1):
3      particles_for_this_node += (total_particles % MPI_WORLD_SIZE)
4  /* Memory allocations for the Particle array based on
5  the total number of particles assigned to this node */
6  Particle *particle_array = malloc(particles_for_this_node)
7  /* Memory allocations for private grids */
8  GridQty1 *private_grid_qty_1 = malloc(GRID_DIMENSIONS)
9  GridQty2 *private_grid_qty_2 = malloc(GRID_DIMENSIONS)
10 ...
11 GridQtyN *private_grid_qty_n = malloc(GRID_DIMENSIONS)
12 /* Other initializations */

```

Once these initialization are complete, each node enters the simulation loop. The pseudo-code for the simulation loop is as follows:

```

1  while (simulation_not_over):
2      /* Execution of charge deposition module,
3      using the thread level parallelization */
4      chargeDeposition();
5      /* Global synchronization of private node-level copies of grid quantities */
6      MPI_Allreduce(private_grid_qty_1, ...)
7      MPI_Allreduce(private_grid_qty_2, ...)
8      ...
9      MPI_Allreduce(private_grid_qty_n, ...)
10     /* After the call to Allreduce, each node has
11     consistent view of grid quantities */
12     /* Execution of mover module using the thread level parallelization */
13     mover(grid_qty)
14     /* Other modules of the PIC simulation */

```

6 Results

As the grid size increases, while keeping the Particle Per Cell (PPC) value constant, then the number of particles also increases. Therefore, the memory consumption also increases for both grid and particle data structures. In the Table 4, we have reported the memory consumption of grid data structures used to store electric and magnetic fields, and the memory consumption of the particle data structure. Similar amount of memory is also used for other grid based data structures such as electron number density, ion number density, charge density, energy and potential. The memory taken by one element of the respective data structure is reported in the Sect. 4.1. It is evident from the first column of the Table 4 that the memory needed to store the array of particles increases drastically as we increase the grid size.

The biggest advantage of our hybrid parallelization strategy is in terms of memory consumption. Since the particles are divided equally among the nodes, even if we increase the grid size, the memory used to store all the particles would increase but it would be less per node when compared to the OpenMP parallelization strategy in which the array of particles would have been stored in just one node. This analysis is shown in the Table 4.

Table 4. Memory consumed in MBs by the particle and grid data structures - Electric Field (EF) and Magnetic Field (MF) per each MPI node - in the parallelization achieved using OpenMP and the hybrid parallelization strategy for different problem size and fixed particle per cell(PPC) value of 80.

Grid size	OpenMP with 4 cores			Hybrid with 4 nodes, 4 core each			Hybrid with 8 nodes, 4 core each		
	Particle array size (MB)	EF array size (MB)	MF array size (MB)	Particle array size (MB)	EF array size (MB)	MF array size (MB)	Particle array size (MB)	EF array size (MB)	MF array size (MB)
512×512	1040	4	2	260	4	2	130	4	2
1024×1024	4160	16	4	1060	16	4	520	16	4
2048×2048	16640	64	16	4160	64	16	2080	64	16

Table 5. Comparison of the execution time (in seconds) taken by the OpenMP code run on 4 cores (Bench 1 of Table 1) and the Hybrid code run on different number of nodes each having some number of cores (Bench 4 of Table 1, using all 4 nodes) for 100 iterations of the simulation with different grid sizes and particle per cell(PPC) values.

Grid size	Run time (sec) for 40 PPC			Run time (sec) for 80 PPC		
	OpenMP 1 node, 4 cores per node	Hybrid 4 nodes, 4 cores per node	Hybrid 8 nodes, 4 cores per node	OpenMP 1 node, 4 cores per node	Hybrid 4 nodes, 4 cores per node	Hybrid 8 nodes, 4 cores per node
512×512	64.26	26.48	18.55	115.97	43.16	25.74
1024×1024	457.36	146.35	92.96	972.47	254.44	157.82
2048×2048	2380.07	709.87	427.65	4770.77	1592.25	681.88

Table 6. Comparison of the speedup for hybrid system with different number of MPI nodes for 100 iterations of the simulation with different grid sizes and particle per cell(PPC) values. Here, Speedup = execution time of OpenMP based code on a multi-core processor with 4 cores (Bench 1)/execution time of corresponding hybrid system (Bench 4)

Grid size	Speedup for 40 PPC		Speedup for 80 PPC	
	Hybrid 4 nodes, 4 cores per node	Hybrid 8 nodes, 4 cores per node	Hybrid 4 nodes, 4 cores per node	Hybrid 8 nodes, 4 cores per node
512×512	2.42	3.46	2.69	4.51
1024×1024	3.13	4.92	3.82	6.16
2048×2048	3.35	5.57	3.00	7.00

As we can see from the Tables 5 and 6, if we keep the number of nodes fixed then the speedup increases when we increase the problem size, i.e., increasing the grid size and/or increasing the particle per cell (PPC). Also, for large problem size, increasing the number of cores/node leads to increase in the speedup indicating the code scales well for architectures with many nodes or cores/node. One reason for this is that as we increase the problem size, the aggregation of the node-level private copy of grid quantities through `MPI_Allreduce` takes negligible time compared to the time taken in performing extensive interpolations for large number of particles per node.

Table 7. Comparison of the execution time (in seconds) of the Mover module for the hybrid code with different number of cores and different problem size and fixed PPC value of 80.

Grid size	Time (sec) – Mover		
	OpenMP	Hybrid	Hybrid
	1 node,	4 nodes,	8 nodes,
	4 cores per node	4 cores per node	4 cores per node
512×512	71.42	17.24	8.66
1024×1024	540.023	113.97	53.90
2048×2048	2768.83	622.98	295.51

Table 8. Speedup of the Mover module for the hybrid code with different number of cores and different problem size and fixed PPC value of 80. Here, Speedup = execution time of OpenMP based code on a multi-core processor with 4 cores (Bench 1)/execution time of corresponding hybrid system (Bench 4)

Grid size	Speedup – Mover	
	Hybrid 16 cores	Hybrid 32 cores
512×512	4.14	8.24
1024×1024	4.73	10.02
2048×2048	4.44	9.37

As shown in the Fig. 3, the *Poisson Solver* and *Collision* modules take negligible fraction of time out of the whole computation time. Therefore, we have only reported the results for the *Mover* module in Tables 7 and 8. As can be seen from the Table 8, for large problem sizes, the *Mover* module give super-linear speedup (marginally), suggesting that they are able to efficiently use the (combined) cache compared to the OpenMP implementation.

Also, from the results of Table 8 one can see that the speedup is maximum when the grid size is 1024×1024 and the speedup for 2048×2048 is lesser than that when grid size is 1024×1024 . One plausible explanation for this is that as we increase the grid size, after a point it is not possible to accommodate the entire Electric and Magnetic field's grid in the cache, as can be observed from the Table 4. For the specific case of 1024×1024 grid size, the grids barely fit in the cache for the hybrid code as well as OpenMP code but the cache utilization for this specific grid size is much more significant for hybrid code than its OpenMP counterpart.

7 Conclusion

We have proposed and investigated in detail an optimized shared memory parallelization strategy of the PIC-MCC algorithm for plasma simulation on Intel Xeon-Phi (many-core) and Xeon (multi-core) architectures as well as an efficient hybrid (OpenMP+MPI) implementation which can be easily implemented on a HPC cluster consisting of nodes with multi-core processors. In order to take advantage of recent advancements in architectures supporting hybrid parallelization, our hybrid parallelization strategy is a natural extension of the shared memory parallelization strategy which requires minimal changes in the code. Our numerical experiments with different problem sizes reveal that for large problem sizes it becomes increasingly difficult to use just the shared memory parallelization strategy because of the limited access to shared primary memory. But the hybrid parallelization strategy makes such large scale simulations possible as it makes use of a large number of nodes such that the particles fit in the primary memory of their respective nodes. Also, unlike other previous strategies based on grid decomposition, our particle decomposition strategy scales very efficiently with increasing number of nodes. We also observe that unlike Xeon multi-core architecture, observed speedup is below expectation in case of Xeon-Phi MIC processors due to lack of scope of vectorization of PIC algorithm and other limiting factors pertaining to serial executions on Xeon-Phi. Efficient hybrid PIC codes opens up many possibilities for research in the area of Low Temperature Plasmas which require computationally intensive PIC-MCC simulations.

Acknowledgement. The work has been carried out using the HPC facilities at DA-IICT and hardware received under BRNS-PFRC project. We would also like to acknowledge the help received from Colfax's remote access program. We thank Sid-darth Kamaria, Harshil Shah and Riddhesh Markandeya for their contribution towards the serial code development. Miral Shah thanks Department of Atomic Energy, Govt. of India for junior research fellowship (JRF) received under BRNS-PFRC project (No. 39/27/2015-BRNS/34081).

References

1. Adamovich, I., et al.: The 2017 plasma roadmap: low temperature plasma science and technology. *J. Phys. D: Appl. Phys.* **50**(32), 323001 (2017)
2. Birdsall, C.K., Langdon, A.B.: *Plasma Physics via Computer Simulations*. CRC Press, Boca Raton (1991)
3. Birdsall, C.K., Fellow, L.: Particle-in-cell charged-particle simulations, plus Monte Carlo collision with neutral atom, PIC-MCC. *IEEE Trans. Plasma Sci.* **19**(2), 65–85 (1991)
4. Shah, H., Kamaria, S., Markandeya, R., Shah, M., Chaudhury, B.: A novel implementation of 2D3V particle-in-cell (PIC) algorithm for Kepler GPU architecture. In: *IEEE 24th International Conference on High Performance Computing (HiPC)*, pp. 378–387 (2017)
5. Verboncoeur, J.P.: Particle simulation of plasmas: review and advances. *Plasma Phys. Control. Fusion* **47**(5A), A231–A260 (2005)
6. Hariri, F., et al.: A portable platform for accelerated PIC codes and its application to GPUs using OpenACC. *Comput. Phys. Commun.* **207**, 69–82 (2016)
7. Derouillat, J., et al.: SMILEI: a collaborative, open-source, multi-purpose particle-in-cell code for plasma simulation. *Comput. Phys. Commun.* **222**, 351–373 (2018)
8. Decyk, V.K., Singh, T.V.: Particle-in-cell algorithms for emerging computer architectures. *Comput. Phys. Commun.* **185**(3), 708–719 (2014)
9. Carmona, E.A., Chandler, L.J.: On parallel PIC versatility and the structure of parallel PIC approaches. *Concurr. Comput.: Pract. Exp.* **9**, 1377–1405 (1997)
10. Adams, M.F., Ethier, S., Wichmann, N.: Performance of particle in cell methods on highly concurrent computational architectures. *J. Phys.: Conf. Ser.* **78**, 012001 (2007)
11. Burau, H., et al.: PIConGPU: a fully relativistic particle-in-cell code for a GPU cluster. *IEEE Trans. Plasma Sci.* **38**(10), 2831–2839 (2010)
12. Claustre, J., Chaudhury, B., Fubiani, G., Paulin, M., Boeuf, J.P.: Particle-in-cell monte carlo collision model on GPU-application to a low-temperature magnetized plasma. *IEEE Trans. Plasma Sci.* **41**(2), 391–399 (2013)
13. Madduri, K., Su, J., Williams, S., Oliker, L., Ethier, S., Yelick, K.: Optimization of parallel particle-to-grid interpolation on leading multicore platforms. *IEEE Trans. Parallel Distrib. Syst.* **23**(10), 1915–1922 (2012)
14. Boeuf, J.P., Chaudhury, B., Garrigues, L.: Physics of a magnetic filter for negative ion sources. I. Collisional transport across the filter in an ideal, 1D filter. *Phys. Plasmas* **19**(11), 113509 (2012)
15. Fehske, H., Schneider, R., Weibe, A.: *Computational Many-Particle Physics. Lecture Notes in Physics*, vol. 739. Springer, Heidelberg (2008). <https://doi.org/10.1007/978-3-540-74686-7>
16. Tskhakaya, D., Matyash, K., Schneider, R., Taccogna, F.: The particle-in-cell method. *Contrib. Plasma Phys.* **47**(8–9), 563–594 (2007)
17. Schenk, O., Gartner, K.: Solving unsymmetric sparse systems of linear equations with PARDISO. *Futur. Gener. Comput. Syst.* **20**(3), 475–487 (2004)
18. Boris, J.P.: Relativistic plasma simulation-optimization. In: *4th Conference on Numerical Simulation of Plasma*, no. November 1970, p. 3 (1970)
19. Lapenta, G.: Particle-based simulation of plasmas. In: *Plasma Modeling*. IOP Publishing, Bristol (2016). <https://doi.org/10.1088/978-0-7503-1200-4ch4>. Chap. 4
20. Sodani, A., et al.: Knights landing: second-generation Intel Xeon Phi product. *IEEE Micro* **36**(2), 34–46 (2016)

21. Bansal, G., et al.: Negative ion beam extraction in ROBIN. *Fusion Eng. Des.* **88**, 778–782 (2013)
22. Rabenseifner, R.: Hybrid parallel programming on HPC platforms. In: Fifth European Workshop on OpenMP, EWOMP 2003, Aachen, Germany, 22–26 September 2003 (2003)
23. Bowers, K.: Accelerating a particle-in-cell simulation using a hybrid counting sort. *J. Comput. Phys.* **173**, 393–411 (2001)
24. Hoefer, T., Belli, R.: Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, p. 73 (2015)